

Amazon Google Hourly Series Forecasting

Project type: Machine Learning (time-series forecasting)

This Google Colab notebook uses the supplied CSV to forecast the next 24 hourly values for one selected time series. The file contains **ds** (timestamp), **y** (observed value), and **unique_id** (series identifier). The source file does not include a lookup table explaining which IDs map to Amazon or Google, so select a series by its numeric ID and verify its meaning from the original dataset documentation before making business claims.

Objective

Build and evaluate a machine-learning model that predicts hourly values from recent history and daily/weekly seasonal patterns. Compare it with a seasonal-naive baseline using a chronological holdout set.

Target

y numeric hourly observation. This is a regression and forecasting task, not NLP or classification.

Method

Lag and rolling-window features + **HistGradientBoostingRegressor**; chronological validation; MAE and RMSE metrics.

1. Load the CSV

Upload the provided CSV when prompted. For repeated use, place it in Google Drive and set **CSV_PATH** to its Drive path instead.

Code cell 3

```
from google.colab import files
from pathlib import Path

uploaded = files.upload()
CSV_PATH = next(iter(uploaded))
print('Using:', CSV_PATH)
```

Output

<IPython.core.display.HTML object>

Output

Saving y_amazon-google-large.csv to y_amazon-google-large.csv
Using: y_amazon-google-large.csv

Code cell 4

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(CSV_PATH)
# The first column in this file is an exported row index, not a model feature.
df = df.rename(columns={df.columns[0]: 'row_index'}) if df.columns[0].startswith('Unnamed:') else
```

```

df
required = {'ds', 'y', 'unique_id'}
missing = required - set(df.columns)
if missing:
    raise ValueError(f'Missing required columns: {missing}')
df['ds'] = pd.to_datetime(df['ds'], errors='coerce')
df['y'] = pd.to_numeric(df['y'], errors='coerce')
df['unique_id'] = pd.to_numeric(df['unique_id'], errors='coerce')
df = df.dropna(subset=['ds', 'y', 'unique_id']).copy()
df['unique_id'] = df['unique_id'].astype(int)
print(f'{len(df):,} rows | {df.unique_id.nunique()} series | {df.ds.min()} through {df.ds.max()}')
display(df.head())

```

Output

3,055,000 rows | 235 series | 2020-01-01 08:00:00 through 2021-06-25 23:00:00

Output

	row_index	ds	y	unique_id
0	0	2020-01-01 08:00:00	6.0	0
1	1	2020-01-01 09:00:00	0.0	0
2	2	2020-01-01 10:00:00	4.0	0
3	3	2020-01-01 11:00:00	8.0	0
4	4	2020-01-01 12:00:00	4.0	0

2. Select a series and prepare features

The notebook models one series at a time so it runs comfortably in Colab and the forecast is easy to interpret. Change **SERIES_ID** to another ID present in the file. Features use only past observations; the last seven days are held out for evaluation.

Code cell 6

```

SERIES_ID = 0 # Change this to a unique_id in the CSV
HORIZON = 24 # Forecast 24 hours
HOLDOUT_HOURS = 24 * 7

series = (df.loc[df['unique_id'] == SERIES_ID, ['ds', 'y']]
          .sort_values('ds').drop_duplicates('ds').set_index('ds').asfreq('h'))
# Fill only timestamp gaps using time interpolation; report how many were introduced.
gaps = int(series['y'].isna().sum())
series['y'] = series['y'].interpolate(method='time').ffill().bfill()
print(f'Series {SERIES_ID}: {len(series):,} hourly points; {gaps} missing timestamps filled.')
if len(series) <= HOLDOUT_HOURS + 168:
    raise ValueError('Selected series is too short for a 7-day holdout and 168-hour lag.')

def make_features(s):
    x = pd.DataFrame(index=s.index)
    x['lag_1'] = s.shift(1)
    x['lag_24'] = s.shift(24)
    x['lag_168'] = s.shift(168)
    x['mean_24'] = s.shift(1).rolling(24).mean()
    x['mean_168'] = s.shift(1).rolling(168).mean()
    x['hour'] = x.index.hour
    x['dayofweek'] = x.index.dayofweek
    x['dayofyear'] = x.index.dayofyear
    return x

features = make_features(series['y'])
data = features.join(series.rename(columns={'y': 'target'})).dropna()
train = data.iloc[:-HOLDOUT_HOURS]
test = data.iloc[-HOLDOUT_HOURS:]
print(f'Train: {train.index.min()} to {train.index.max()} ({len(train)} rows)')
print(f'Test: {test.index.min()} to {test.index.max()} ({len(test)} rows)')

```

Output

Series 0: 13,000 hourly points; 0 missing timestamps filled.
Train: 2020-01-08 08:00:00 to 2021-06-18 23:00:00 (12664 rows)
Test: 2021-06-19 00:00:00 to 2021-06-25 23:00:00 (168 rows)

3. Train and evaluate the ML model

The seasonal-naive baseline predicts each hour using the value from 24 hours earlier. The ML model is evaluated on exactly the same final seven-day period. No random shuffle is used, which preserves time order.

Code cell 8

```
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

FEATURE_COLS = features.columns.tolist()
model = HistGradientBoostingRegressor(max_iter=150, learning_rate=0.08, max_leaf_nodes=15,
                                     random_state=42)
model.fit(train[FEATURE_COLS], train['target'])
pred = np.maximum(0, model.predict(test[FEATURE_COLS]))
actual = test['target'].to_numpy()
naive = test['lag_24'].to_numpy()

def score(y_true, y_pred):
    return {'MAE': mean_absolute_error(y_true, y_pred),
            'RMSE': mean_squared_error(y_true, y_pred) ** 0.5}

results = pd.DataFrame({'Seasonal naive (24h)': score(actual, naive),
                       'Gradient boosting': score(actual, pred)}).T
display(results.round(3))
```

Output

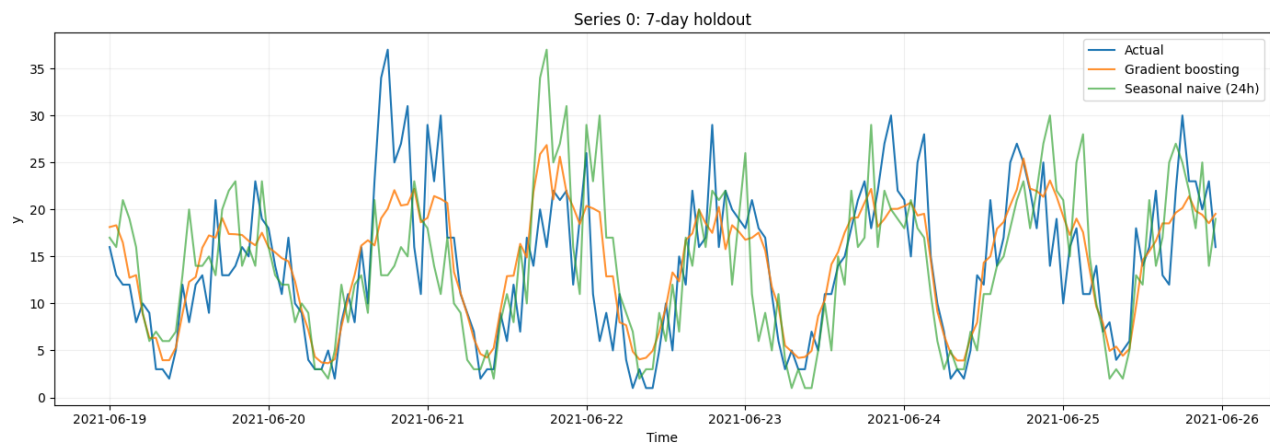
	MAE	RMSE
Seasonal naive (24h)	5.411	7.118
Gradient boosting	3.650	4.820

Code cell 9

```
plt.figure(figsize=(14, 5))
plt.plot(test.index, actual, label='Actual', linewidth=1.5)
plt.plot(test.index, pred, label='Gradient boosting', alpha=0.85)
plt.plot(test.index, naive, label='Seasonal naive (24h)', alpha=0.65)
plt.title(f'Series {SERIES_ID}: 7-day holdout')
plt.xlabel('Time'); plt.ylabel('y'); plt.legend(); plt.grid(alpha=.2); plt.tight_layout();
plt.show()
```

Output

<Figure size 1400x500 with 1 Axes>



4. Forecast the next 24 hours

Forecast recursively: each predicted value becomes part of the history used to predict the next hour. These are point forecasts; no uncertainty intervals are calculated.

Code cell 11

```

history = series['y'].copy()
future_rows = []
for _ in range(HORIZON):
    next_time = history.index[-1] + pd.Timedelta(hours=1)
    x_next = make_features(history).iloc[[-1]].copy()
    yhat = max(0.0, float(model.predict(x_next[FEATURE_COLS])[0]))
    future_rows.append((next_time, yhat))
    history.loc[next_time] = yhat
forecast = pd.DataFrame(future_rows, columns=['ds', 'forecast']).set_index('ds')
display(forecast)
plt.figure(figsize=(12, 4))
plt.plot(series.index[-7*24:], series['y'].iloc[-7*24:], label='Recent actuals')
plt.plot(forecast.index, forecast['forecast'], marker='o', label='24-hour forecast')
plt.title(f'Next 24 hours ? series {SERIES_ID}')
plt.xlabel('Time'); plt.ylabel('y'); plt.legend(); plt.grid(alpha=.2); plt.tight_layout();
plt.show()

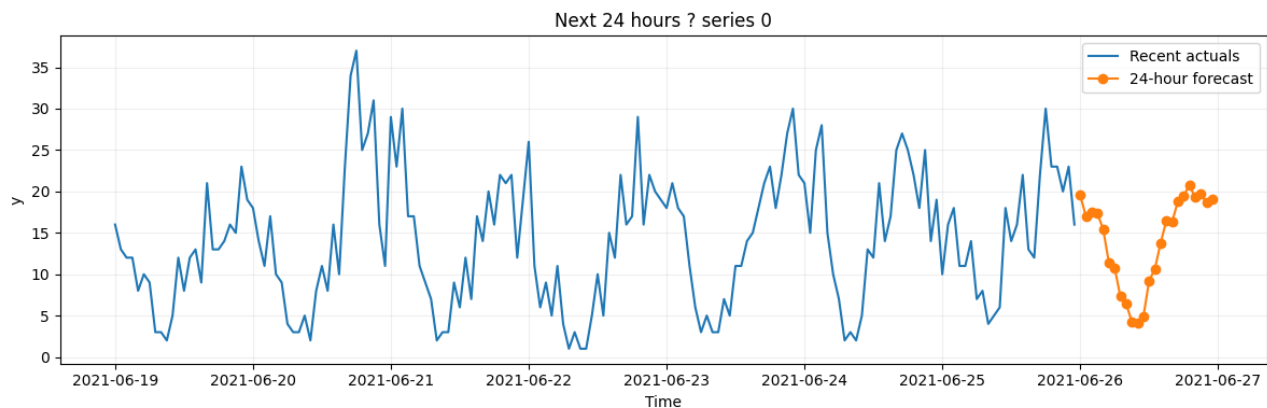
```

Output

ds	forecast
2021-06-26 00:00:00	19.534123
2021-06-26 01:00:00	17.021866
2021-06-26 02:00:00	17.549865
2021-06-26 03:00:00	17.345612
2021-06-26 04:00:00	15.454769
2021-06-26 05:00:00	11.404607
2021-06-26 06:00:00	10.797353
2021-06-26 07:00:00	7.296106
2021-06-26 08:00:00	6.413264
2021-06-26 09:00:00	4.278845
2021-06-26 10:00:00	4.085976
2021-06-26 11:00:00	4.843391
2021-06-26 12:00:00	9.182712
2021-06-26 13:00:00	10.639945
2021-06-26 14:00:00	13.746520
2021-06-26 15:00:00	16.464966
2021-06-26 16:00:00	16.302481
2021-06-26 17:00:00	18.755266
2021-06-26 18:00:00	19.397369
2021-06-26 19:00:00	20.790741
2021-06-26 20:00:00	19.309570
2021-06-26 21:00:00	19.697535
2021-06-26 22:00:00	18.646905
2021-06-26 23:00:00	19.027547

Output

<Figure size 1200x400 with 1 Axes>



Conclusion

This project applies supervised machine learning to hourly time-series forecasting. It compares a gradient-boosting model against a 24-hour seasonal-naïve baseline on a chronological seven-day holdout. Use the MAE/RMSE table and plot above to determine whether the model improves on the baseline for the selected series. Results apply only to the selected `unique_id` and historical period; they do not by themselves establish that the series represents a particular company or search metric.

Future scope

- Confirm the meaning and source of each `unique_id`, then report results for both Amazon and Google only if the mapping is documented.
- Evaluate every series and aggregate results by series instead of reporting one selected ID.
- Test rolling-origin backtesting across multiple dates and forecast horizons.
- Compare additional models such as LightGBM/XGBoost, SARIMA, Prophet, or a global multi-series model.
- Add prediction intervals and monitor error/drift as new observations arrive.
- Include external signals (holidays, promotions, trends, or exogenous variables) when available.